

dslrBooth Remote Integration – Technical Overview

This document describes an application designed to integrate with the dslrBooth software in order to remotely start and end photo booth sessions from another machine. The solution leverages dslrBooth's built-in API and trigger mechanisms, combined with a custom Webhook API and a lightweight controller application.

1. Objective

The primary goal of this integration is to allow a remote system (for example, a control panel, kiosk, or operator workstation) to start and end dslrBooth sessions without directly interacting with the dslrBooth user interface.

2. Architecture Overview

The integration consists of three main components:

1. dslrBooth Software – Installed on the photo booth machine.
2. Webhook API Service – A local or network-accessible API that receives events and commands.
3. Remote Controller Application – A UI or service running on another machine that triggers sessions.

4. Technologies Used

- .Net Core / C#
- Visual Studio
- Log4net
- API
- RequestCatcher
- Postman

3. dslrBooth Configuration

Within dslrBooth, the Triggers and API features are enabled. When specific events occur (such as session start or end), dslrBooth can call a configured URL and pass the event type as a GET parameter.

Example Trigger URL:

`http://localhost:5161/api/Event`

This allows the external application to be notified whenever dslrBooth changes state.

4. Webhook API

A custom Webhook API is exposed using a REST interface. This API is documented using Swagger UI for easy testing and visibility.

Key endpoints include:

- GET /api/Event – Receives event callbacks from dslrBooth.
- GET /api/Session/start – Initiates a new dslrBooth session.
- GET /api/Session/end – Ends the current dslrBooth session.

5. Session Control Flow

1. The remote controller application sends a request to /api/Session/start.
2. The Webhook API validates the request and calls the dslrBooth API with the correct parameters.
3. dslrBooth starts a session and displays the session UI (dslrbooth Home Screen).
4. When the session is complete, dslrbooth sends a request to /api/Session/end.
5. dslrBooth ends the session and begins processing and printing.

6. Remote Controller Application

The remote controller provides a simple user interface with Start and End buttons. This UI abstracts all technical details and allows an operator to control sessions from another machine on the same network.

7. Logging and Monitoring

The Webhook API produces log files for both informational messages and errors. These logs are used to verify session flow, diagnose failures, and ensure reliable operation during live events.

8. Benefits

- Remote control of dslrBooth sessions
- Reduced need for direct operator interaction
- Easier integration with custom hardware or software
- Clear API documentation via Swagger
- Improved reliability through logging and monitoring

9. Conclusion

This integration provides a robust and extensible approach to controlling dslrBooth remotely. By combining dslrBooth's API capabilities with a custom Webhook service and controller UI, operators gain greater flexibility and automation for professional photo booth deployments.

10. Webhook Testing with Request Catcher

During development and testing, RequestCatcher can be used to validate webhook calls sent from dslrBooth. It provides a temporary public URL that captures and displays all incoming HTTP requests in real time.

This is especially useful when testing event triggers without deploying the Webhook API to a public server. dslrBooth can be configured to send trigger events directly to a RequestCatcher URL to confirm payload structure, query parameters, and timing.

11. Event Delivery and Sharing Screen Flow

After a session completes, dslrBooth transitions to the sharing screen. At this stage, users may choose to share their content via Photo, GIF, Email, SMS, or QR code, depending on the configuration.

If the booth or sharing service is temporarily unavailable, dslrBooth displays a status message indicating that the service is closed and will resume shortly. These states can also be reported back to the Webhook API as events for monitoring or analytics purposes.

12. Extended Use Cases

By combining remote session control with webhook event tracking, the system can be extended to support:

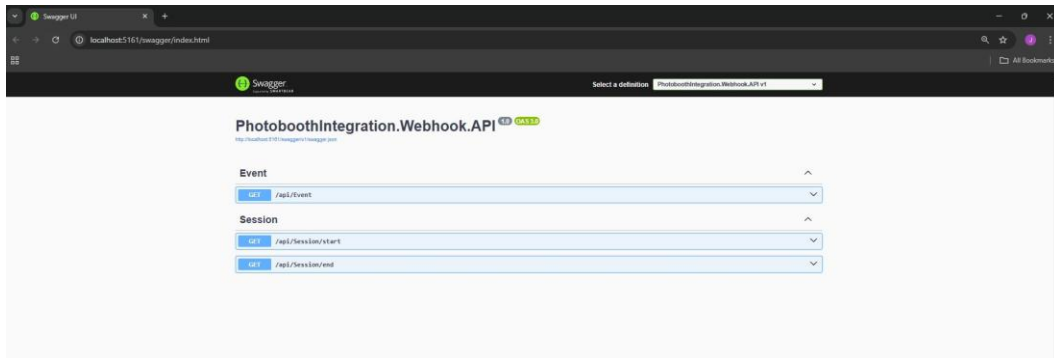
- Automated session start based on external signals
- Centralized monitoring dashboards
- Integration with CRM or cloud storage systems
- Real-time sharing and delivery tracking
- Failover and service availability notifications

13. Final Notes

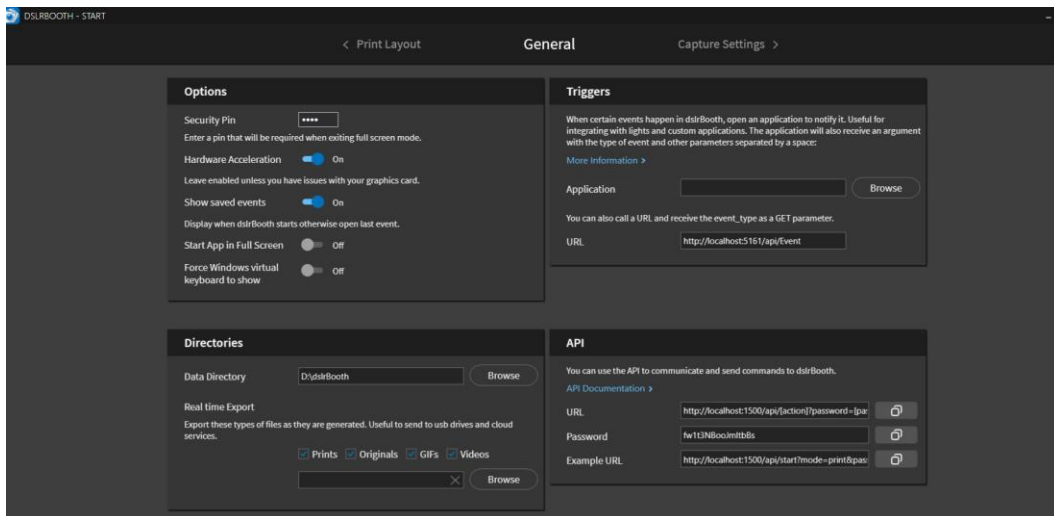
This enhanced integration demonstrates how dslrBooth can be controlled, monitored, and extended using standard web technologies. The addition of webhook testing tools and sharing-state awareness improves reliability and confidence during live deployments.

Appendix A – Reference Screenshots

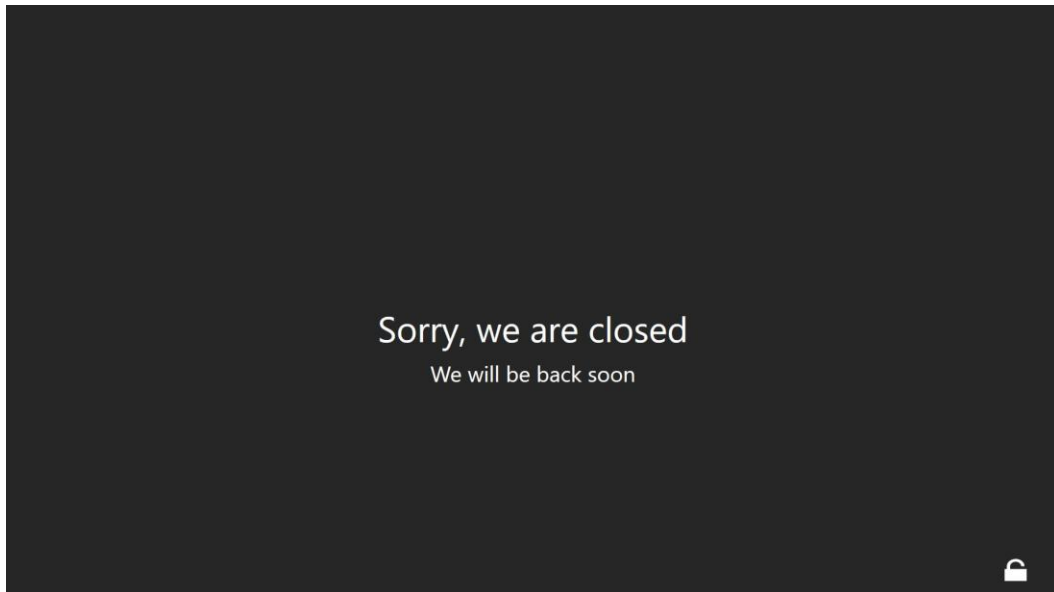
Swagger UI – Webhook API endpoints



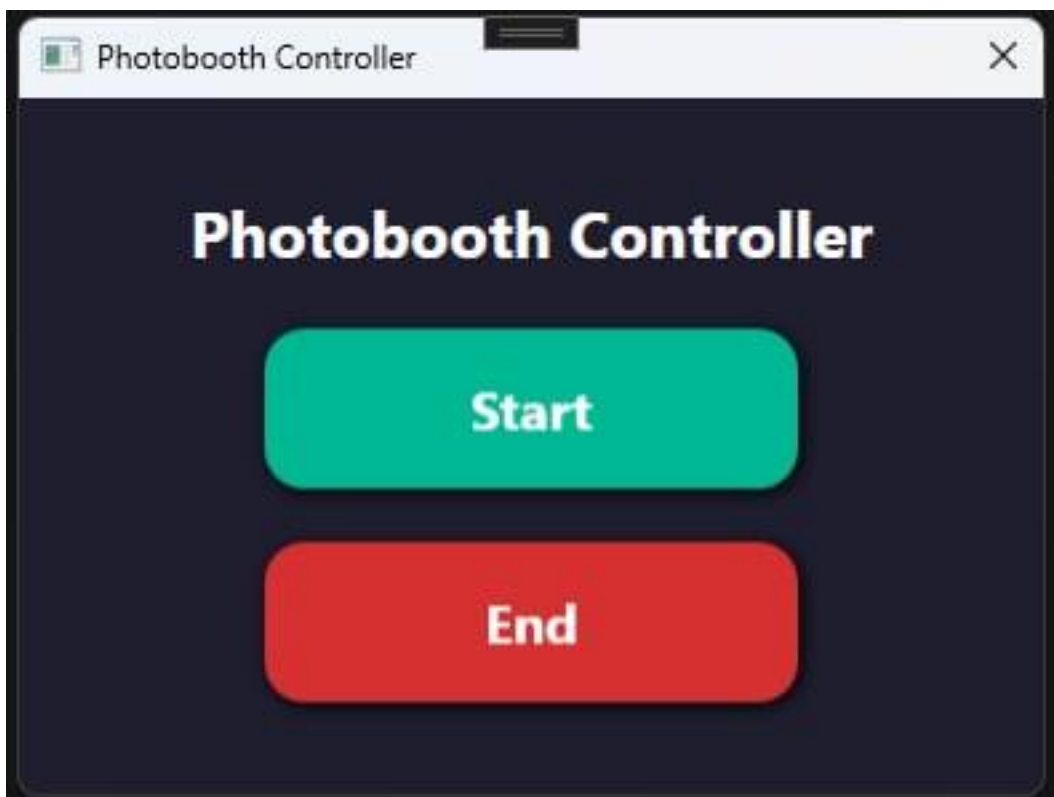
dslrBooth General Settings and Triggers



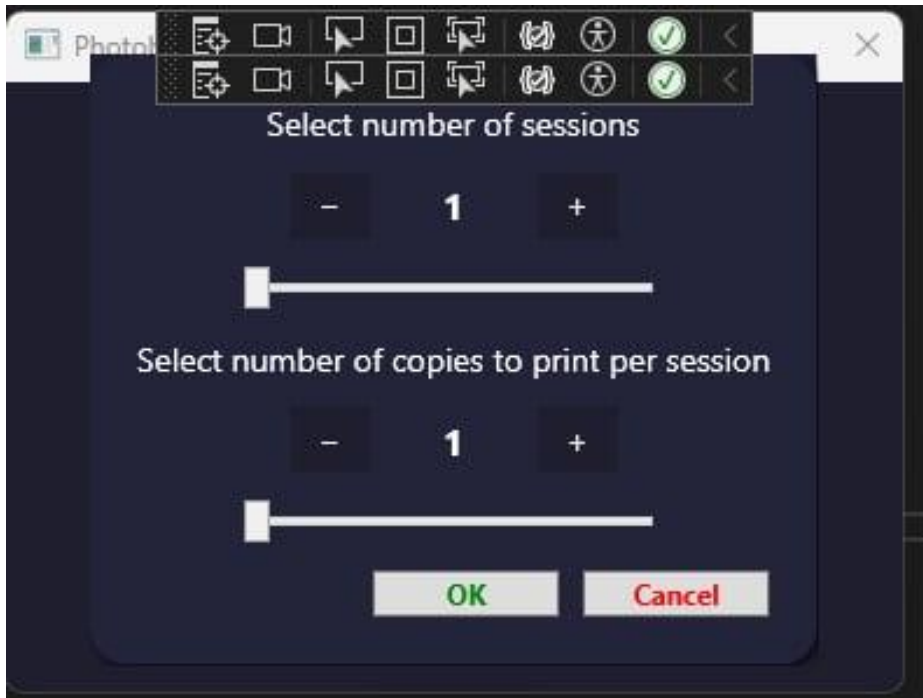
Service Closed / Maintenance Screen



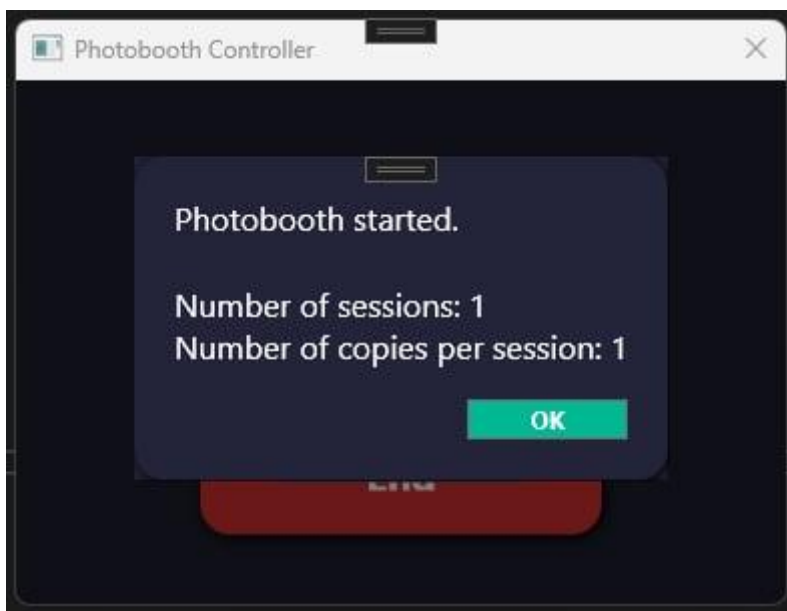
Photobooth Controller – Start/End UI



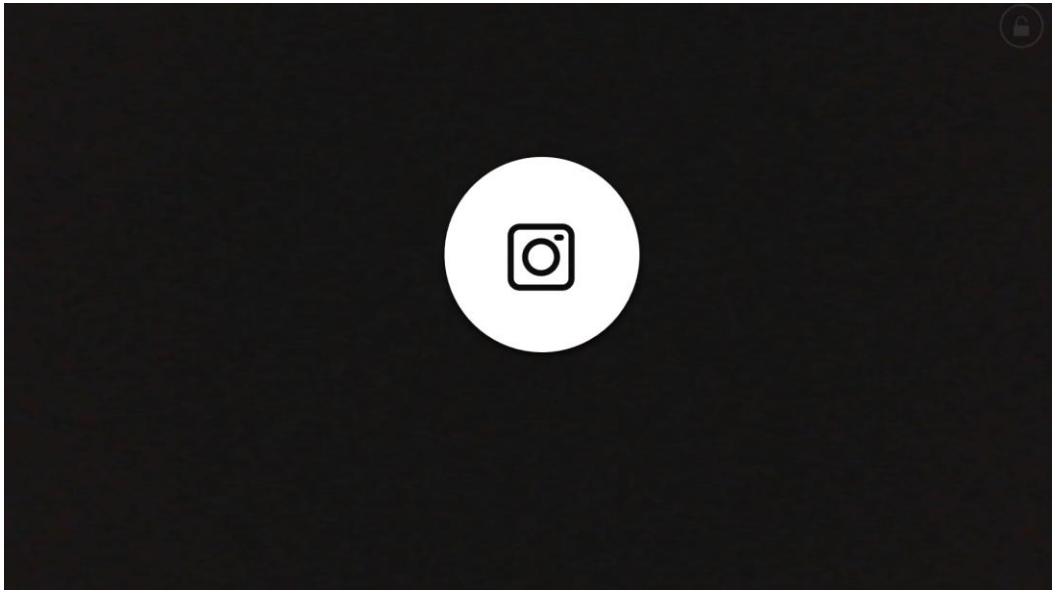
Session Configuration – Number of Sessions and Copies (after clicking Start)



Session Started Confirmation



dslrBooth Home Screen (after start successfully processed)



Template Selection Screen (after clicking the button from Home Screen)

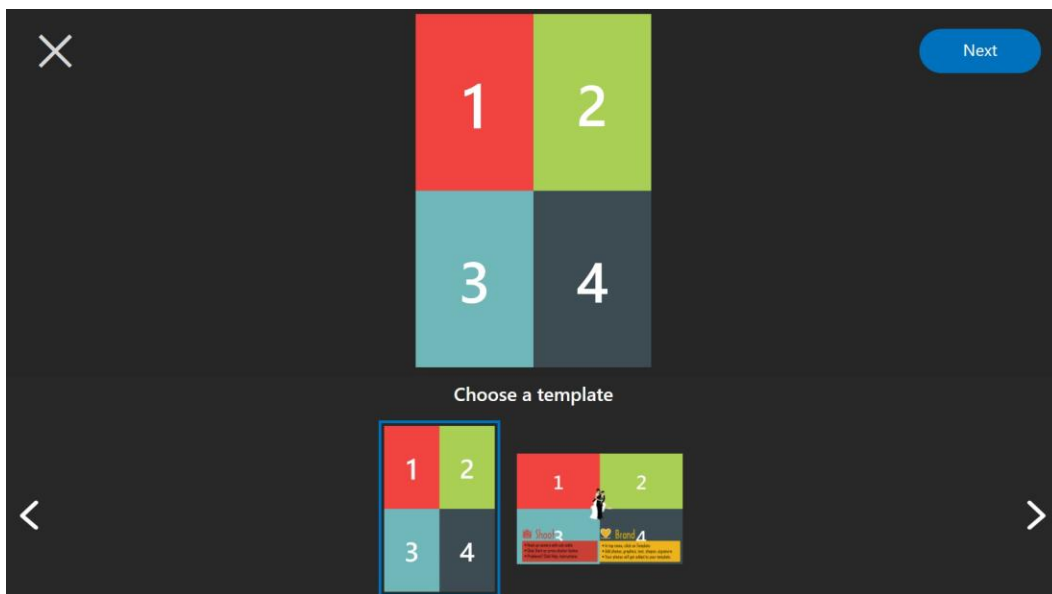
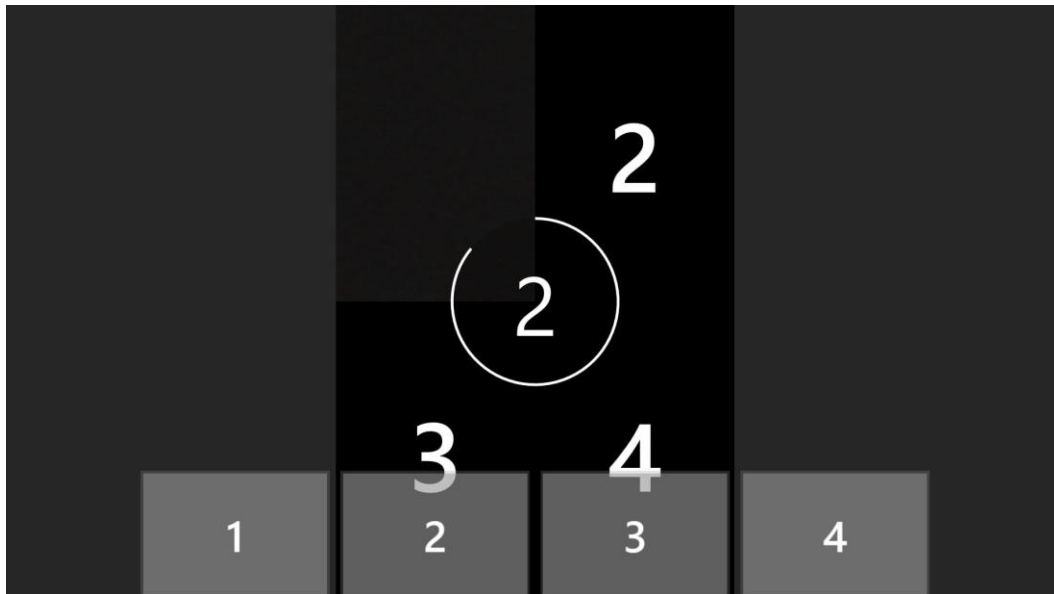
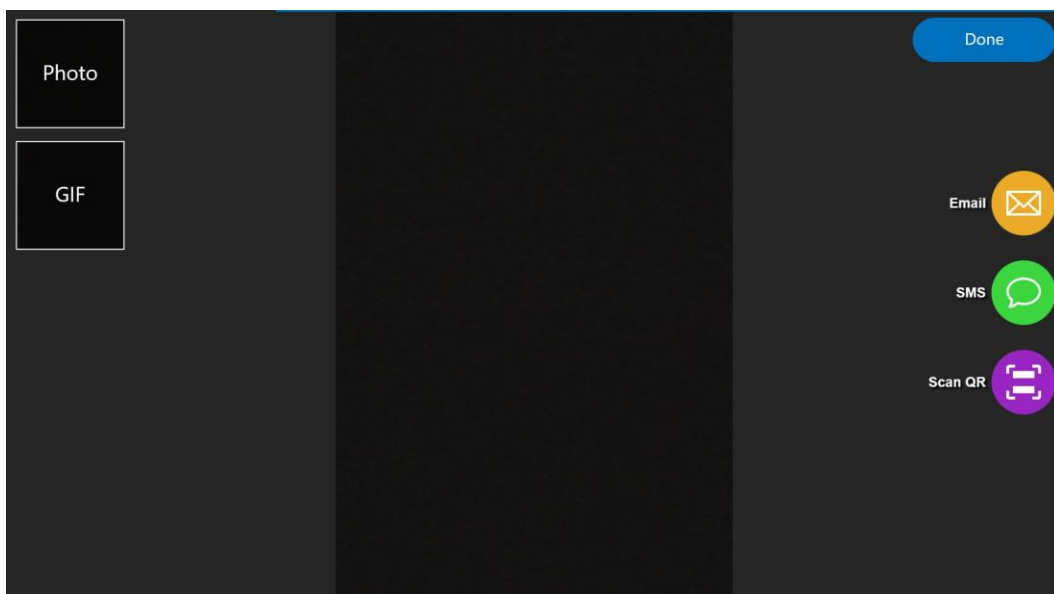


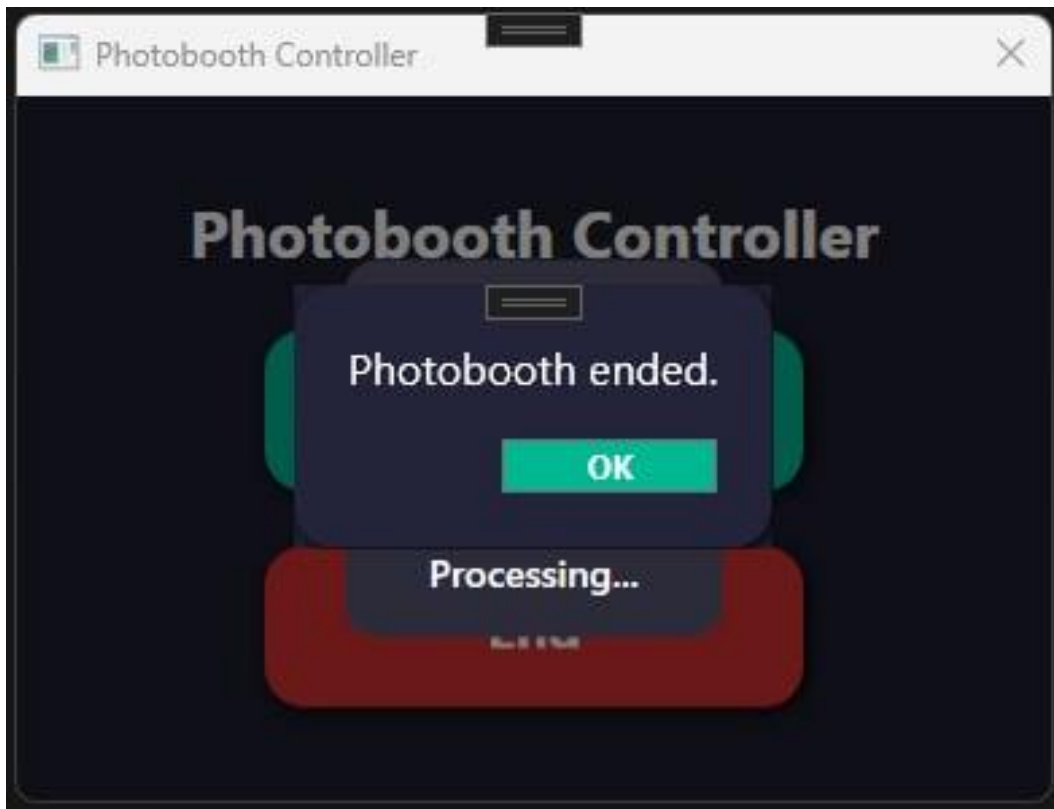
Photo Capture Countdown and Layout (after choosing template)



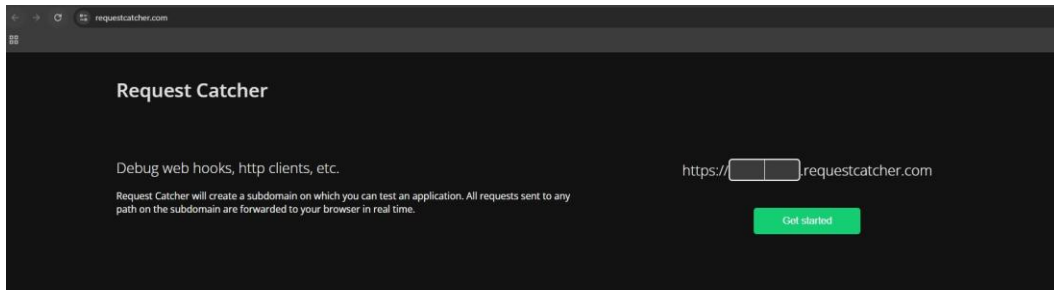
dslrBooth Sharing Options Screen (after taking photos)



Photobooth Session End Confirmation (if manually click the End button – it will go back to the Service Closed / Maintenance Screen)



RequestCatcher – Webhook Testing Interface



Information & Error Logs

... Jiven Files > Projects > Personal Projects > PhotoboothIntegration > PhotoboothIntegration.Webhook > Photobooth.Webhook				
📁 🗑️ ⬆️ Sort ▾ ≡ View ▾ ⋮				
Name	Date modified	Type	Size	
📄 log-error-20251202.txt	02/12/2025 9:07 am	Text Document	5 KB	
📄 log-info-20251202.txt	02/12/2025 9:07 am	Text Document	295 KB	

Appendix B – Figure Index

1. Figure 1 – Swagger UI showing Webhook API endpoints
2. Figure 2 – dslrBooth General Settings and Trigger configuration
3. Figure 3 – Service unavailable / closed screen
4. Figure 4 - Remote Photobooth Controller application (Start / End)
5. Figure 5 – Session configuration dialog (sessions and copies)
6. Figure 6 – Session started confirmation
7. Figure 7 – Home Screen
8. Figure 8 - Template selection screen
9. Figure 9 – Photo capture and countdown sequence
10. Figure 10 – Sharing screen (Photo, GIF, Email, SMS, QR)
11. Figure 11 – Session end confirmation
12. Figure 12 – RequestCatcher webhook testing interface
13. Figure 13 – Application logs for monitoring and diagnostics

Appendix C – Detailed Implementation Notes

This section provides deeper technical clarification and directly references figures shown earlier in this document to support implementation and troubleshooting.

Session Start Sequence

When the remote operator presses the Start button in the Photobooth Controller (see Figure 4), the controller sends a GET request to the Webhook API endpoint `/api/Session/start` (see Figure 1). The Webhook API then forwards the command to the dslrBooth API using the configured password and action parameters.

Session Runtime and Capture

Once the session begins, dslrBooth displays the session configuration dialog (Figure 6), followed by Home Screen (Figure 7). The live capture process, including countdown and frame progression, is shown in Figure 6.

Session End and Processing

Ending the session from the controller (Figure 4) calls the `/api/Session/end` endpoint. dslrBooth confirms the session closing (Figure 11) and proceeds to navigate to the Closed screen (Figure 3). This is not necessary to be included in the workflow as the dslrbooth automatically calls the `/api/Session/end` endpoint after printing the photo (if the session count has reached).

Sharing and Delivery

After processing, the sharing screen (Figure 10) allows guests to receive their content via Email, SMS, QR code, or local export along with printing of photos and ending the session automatically (if the session count has reached).

Webhook Validation and Diagnostics

Webhook calls can be validated during development using RequestCatcher (Figure 12). This confirms query parameters, timing, and event types. Operational logs (Figure 13) provide insight into errors, API calls, and session flow.

Sequence Diagram (Logical Flow)

Remote Controller → Webhook API → dslrBooth API → dslrBooth UI → Webhook Events → Logs / Monitoring

This sequence ensures that all session actions are traceable and recoverable, making the solution suitable for unattended or high-volume environments.

Production Recommendations

- Use HTTPS for all webhook endpoints
- Restrict API access by IP or network segment
- Store API passwords securely
- Enable verbose logging during events

- Test triggers using RequestCatcher before live deployment
- Provide a fallback manual control option on the booth machine

Final Summary

With all components combined, this integration delivers a professional, reliable, and scalable method for remotely controlling dslrBooth sessions. The architecture supports automation, monitoring, and future expansion while remaining simple for operators to use.